

The Prime Memory System for Array Access

DUNCAN H. LAWRIE, SENIOR MEMBER, IEEE, AND CHANDRA R. VORA, MEMBER, IEEE

Abstract—In this paper we describe a memory system designed for parallel array access. The system is based on the use of a prime number of memories and a powerful combination of indexing hardware and data alignment switches. Particular emphasis is placed on the indexing equations and their implementation.

Index Terms—Array access, Burroughs Scientific Processor (BSP), conflict-free array memory, memory system, parallel computer system, SIMD computer memory.

I. INTRODUCTION

THERE have always been certain application areas which require computation power far beyond what technology can deliver, despite the remarkable technological advances made in the past. Today these applications include airframe design, weather prediction, seismic analysis, and physics research, to name just a few. In order to achieve this computation power, computer designers have had to rely on innovative computer organization. The main approach taken in the 1960's and 1970's was to exploit vector processing rather than to only increase the speed of scalar computation. This was done by using either pipeline or parallel computer structures.

Pipeline computers have been the primary commercially successful approach to date, as evidenced by the CRAY-1. These organizations achieve their performance by breaking each basic operation (multiply, add, etc.) into a number of segments. For example, addition can be divided into: 1) mantissa alignment, 2) one or more steps of mantissa addition, and 3) rounding and post normalization. A series of additions can then be sequenced through these segments in less time than it would take to do each addition separately.

A parallel processor consists of a large number of arithmetic units combined into a single machine sharing a common memory system. Theoretically, the performance improvement is limited only by the amount of inherent parallelism in the application rather than by the number of processors (or segments) that can be built.

In practice, however, a number of factors have contributed to the less than optimal performance of parallel machines. One factor is the design of a memory system (a problem shared by high-speed pipeline designs). Given that the processor ensemble is capable of very high computation speeds, a memory

system must be capable of supporting these speeds. This is done by providing parallel (i.e., interleaved) memories. If the data for each parallel operation are evenly and completely distributed over M memories, then all memories can deliver a word during each memory cycle, and the design problem is simply to supply enough memories to provide sufficient bandwidth to match the processors' requirements. But if the data are not evenly distributed, significant degradation can occur due to memory access contention.

The problem discussed in this paper is the design of a memory system that can access, in parallel, the required sections of an array, e.g., a row, column, diagonal, etc. A number of these memory systems have been discussed in the literature. In [2] Batcher discusses a scheme for allowing access to words, bit slices, or "byte" slices of a two-dimensional bit array. Feng described another scheme for accessing various "slices" of data in [7]. Other work described in [20], [21], [10], [11], [14], [18], [13], and [16] has also treated the problem of array access and alignment, but because these designs use a power of two memories and restricted alignment networks, they all have restrictions either on the kinds of "slices" available without memory access conflicts or in the data alignment capabilities. (See [18] for a survey of this work.)

In [3] Budnik and Kuck observed that if the number of memory modules is a prime number, then access to any "linear" array slices can be achieved without conflict (provided that the memory ordering of the desired array elements is relatively prime to the number of array elements). For example, assume that we want to access every other element of a column of an array stored in column major order. If the number of memories M is a power of two, then two parallel accesses would be required to access M elements. But if M is a prime number (>2), only one parallel access would be required (see Fig. 2). This observation turns out to be quite useful. However, the problem of addressing this type of memory turns out to be difficult due to the need to do integer divisions and modulo operations in the addressing hardware. In this paper we will discuss these problems in more detail, and will present a feasible implementation of the prime memory system.

Since many of the ideas in this paper have been incorporated in the design of the Burroughs Scientific Processor (BSP), we will describe some of the details of the memory, alignment, and indexing hardware of this machine. The BSP is a high performance computer designed to be especially effective on vector processing applications, without significantly impairing its performance on scalar computations. A complete description of the BSP is beyond the scope of this paper. However, in

Manuscript received July 2, 1981; revised December 16, 1981. This work was supported by the Burroughs Corporation, Paoli, PA, the Department of Computer Science, University of Illinois at Urbana-Champaign, and the National Science Foundation under Grant US NSF MCS77-27910.

D. H. Lawrie is with the Department of Computer Science, University of Illinois at Urbana-Champaign, Urbana, IL 61801.

C. R. Vora was with the Burroughs Corporation, Paoli, PA 19301. He is now with the Data General Corporation, Westboro, MA 01581.

order to illustrate the ideas in this paper, we will present a brief overview of the architecture. As can be seen in Fig. 1, the BSP consists of 16 processing units, 17 memories, two alignment networks, and a central control and scalar processing unit. Total memory can be as large as eight million words. The control unit includes a fully functional scalar processing unit which can be overlapped with vector operations, and additional memory (256K max words) for scalar data and program storage. Additional components of the system include a high-speed file memory, disks, and a front-end computer. (See [9] for further details.)

The BSP is designed to facilitate vector operations utilizing the 16 processing elements. Special hardware is included in the control unit to perform vector addressing and alignment control, and these operations can be overlapped with vector and scalar processing.

The alignment networks shown in Fig. 1 are in reality crossbar switches controlled by *source tags*. (That is, each output port of the network can supply a "tag" which specifies the number of the input from which it needs data.) While in general, crossbar switches are too expensive for large arrays of processors, due to the relatively small number of processors in the BSP it was determined that crossbar switches were the most cost-effective form of switch capable of performing all the designed alignments.

In particular, the functions such as compress, expand, and merge, require a random aligning pattern which only the crossbar switch could perform efficiently in the allocated time. Other forms of switches were investigated, e.g., the Swanson networks [21], Omega switch [13], Barrel Shift network, etc., but these switches do not perform all the functions needed in the allocated time, and they become cost-effective only with a large number of ports.

II. THE STORAGE SCHEME AND ASSOCIATED EQUATIONS

By a storage scheme we mean the set of rules which determine the module number and address within that module where a given array element is stored. For the present, we will restrict our attention to two-dimensional arrays. However, generalization of these storage schemes is trivial for higher dimensioned arrays.

Fig. 2 shows an 8×8 array stored in 5 memory modules using column major storage. (In order to simplify the equations below, we assume that all arrays are indexed starting at zero unless indicated otherwise.) Notice that any 5 consecutive elements of a row, column, diagonal, etc., all lie in separate modules and thus can be accessed in parallel, i.e., without conflict. For example, the second through sixth elements of the first row are stored in module numbers 3, 1, 4, 2, 0, and at addresses 2, 4, 6, 8, 10, respectively.

We begin with some definitions. Let M be the number of memory modules and P be the number of processors, where we assume $P \leq M$ and M is prime. There are two *storage equations* $f(i, j)$ and $g(i, j)$ which determine the module number and address, respectively, of element (i, j) of the array. In our case we have the following equations:

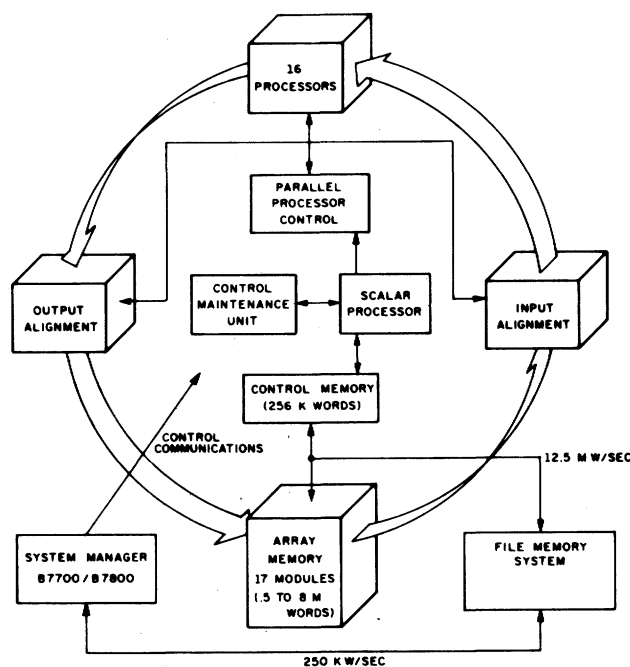


Fig. 1. Block diagram of the Burroughs Scientific Processor.

Memory Module Number

Address	0	1	2	3	4
0	<u>00</u>	10	20	30	x
1	50	60	70	x	40
2	21	31	x	<u>01</u>	11
3	71	x	41	51	61
4	x	<u>02</u>	12	22	32
5	42	52	62	72	x
6	13	23	33	x	<u>03</u>
7	63	73	x	43	53
8	34	x	<u>04</u>	14	24
9	x	44	54	64	74
10	<u>05</u>	15	25	35	x
11	55	65	75	x	45
12	26	36	x	06	16
13	76	x	46	56	66
14	x	07	17	27	37
15	47	57	67	77	x

Fig. 2. Example of a 8×8 array stored column major in 5 memory modules.

$$f(i, j) = [j * I + i + \text{base}] \bmod M \quad (1)$$

$$g(i, j) = [j * I + i + \text{base}] / P \quad (2)$$

where we assume the array is dimensioned (I, J) , "base" is the base address of the array, and P the number of processors is the greatest power of two less than M . Notice that these equations require a mod M operation where M is a prime number. They also require an integer divide by P operation. However, P is a power of two which makes this divide easily implementable. This simplification is made possible by the "holes" shown in Fig. 2. (A hole is an unused memory location.)

Clearly, the number of holes in each row of the memory is equal to $M - P$ in general. For example, if $M = 37$ and $P = 32$, then $5/37$ of the memory is wasted. These holes could be filled with other data, e.g., scalar data, but a cleaner solution is available at the expense of an increase in the complexity of the indexing equations. The necessary modifications will become apparent in Section III when we discuss the partitioning of the equations and the indexing hardware.

Next we define a *linear P-vector* or simply a *P-vector*, to be a *P* element set of the elements of the array formed by linear subscript equations

$$V(a, b, c, e) = \{A(i, j) : i = ax + b, \\ j = cx + e, \quad 0 \leq x < P \leq M\} \quad (3)$$

where again we assume the array is dimensioned $A(I, J)$. Thus, if $a = b = 0$ and $c = e = 1$, then the P -vector ($P = 5$) is the second through sixth elements of the first row of A : $A(0, 1)$, $A(0, 2), \dots, A(0, 5)$. If $a = c = 2$ and $b = e = 0$, then the P -vector ($P = 4$) is every other element of the main diagonal of A : $A(0, 0)$, $A(2, 2), \dots, A(6, 6)$. Notice that the elements of the P -vector are ordered with index x .

Next we define the *index equations* for the P -vector V . We define $\alpha(x)$ to be the address in module $\mu(x)$ of the x th element of the P -vector. Thus, combining (1)–(3) above, we get

$$\begin{aligned}\mu(x) &= f(ax + b, cx + e) \\ &= [(cx + e) * I + (ax + b) + \text{base}] \bmod M \\ &= [dx + B] \bmod M\end{aligned}\quad (4)$$

where $d = a + cI$ and $B = b + eI + \text{base}$. We define d to be the *order* of the P -vector and B to be the initial or base address. Next, we get

$$\begin{aligned}\alpha(x) &= g(ax + b, cx + e) \\ &= [(cx + e) * I + (ax + b) + \text{base}] / P \\ &= [dx + B] / P.\end{aligned}\quad (5)$$

It is easy to show that if d is relatively prime to the number of memory modules, then access to the P -vector can be made without memory conflict. (See [3] and [13] for a proof.)

Since it is most convenient to be able to generate the address $\alpha(x)$ in memory $\mu(x)$, we solve for x in terms of μ and get

$$x(\mu) = [(\mu - B)d'] \bmod M \quad (6)$$

where $d \cdot d' = 1 \pmod{M}$. Substituting this into (5), we get

$$\begin{aligned}\alpha(\mu) &= \{(a + Ic)[(\mu - B)d' \bmod M] + b + eI + \text{base}\}/P \\ &= \{d[(\mu - B)d' \bmod M] + B\}/P.\end{aligned}\quad (7)$$

For example, consider the 5-vector $V(0, 0, 1, 1)$, i.e., the second through sixth elements of the first row of $A(8 \times 8)$. We have $B = 8$ and $d = 8$; thus

$$\begin{aligned}\mu(x) &= [(x) * 8 + 8] \bmod 5 \\ \alpha(x) &= [(x) * 8 + 8] / 4\end{aligned}$$

and since $d' = 2$ (i.e., $2 * 8 = 1 \bmod 5$), $B = 8$, and $M = 5$, we get

$$\alpha(\mu) = \{8[2(\mu - 8) \bmod 5] + 8\}/4.$$

Thus,

$$\begin{aligned}\mu(x) &= (3, 1, 4, 2, 0) \\ \alpha(x) &= (2, 4, 6, 8, 10)\end{aligned}$$

and

$$\alpha(\mu) = (10, 4, 8, 2, 6).$$

Notice that the proper addresses in memories 0, 1, $\dots$, 4, are 10, 4, 8, 2, 6, respectively. We use the $\mu(x)$ equation in the x th processor to determine the module number of the memory containing the x th element of the desired N -vector. At the same time, addressing hardware in memory μ uses the $\alpha(\mu)$ equation to determine the necessary address of the desired element. We use $\alpha(\mu)$ instead of $\alpha(x)$ because this eliminates the need to route the addresses from the processors through the switch.

This process is reasonably straightforward, except that it is not obvious that the hardware can do the necessary calculations efficiently. In Section III we will describe how we partition the equations into parts that can be done separately by special hardware in the CU, AU, and memory addressing box.

III. INDEXING HARDWARE

The BSP consists of a control unit, 16 processors, 17 memories, and two alignment networks as shown in Fig. 1. Vector instructions in the BSP are designed to allow processing on vectors of arbitrary length. The control unit automatically sequences vector operations as a series of *superword* operations where a superword consists of 16 or less vector elements. For example, a vector instruction which specifies a vector of length 53 would be sequenced as three superwords of 16 elements, followed by a superword of 5 elements.

Associated with every array is an *array descriptor* (AD), shown in Fig. 3(a). The two values in the AD describe the base address and total volume (words) of the array and are used for addressing and bounds checks on the array. Every vector instruction refers to at least one and as many as six *vector operands*. Each vector operand is referenced through a *vector set descriptor* (VSD), shown in Fig. 3(b). The VSD actually describes a set of vectors from a given array. B is the address of the first element of the first vector in the set. This vector is ordered with distance d and contains L elements. The first element of the second vector in the set is the (signed) distance D from the first element of the first vector. There are K vectors in the set. Thus, the VSD describes a two-dimensional set of data.

For example, the VSD ($B = 1, d = 8, LL = 8, D = 2, K = 4$) describes the odd-numbered rows of the array $A(8, 8)$ shown in Fig. 2. Similarly, VSD ($B = 0, d = 1, LL = 8, D = 16, K = 4$) describes even numbered columns and VSD ($B = 0, d = 0, LL = 8, D = 1, K = 8$) describes a two-dimensional set of data $X(i, j)$, where $X(i, j) = A(i, 0)$, $0 \leq i, j < 8$, and $A(i, j)$ is the array shown in Fig. 2.

The above parameters are not all stored together. The first step in preparing a vector instruction is to compute the above parameters, together with other values needed for addressing

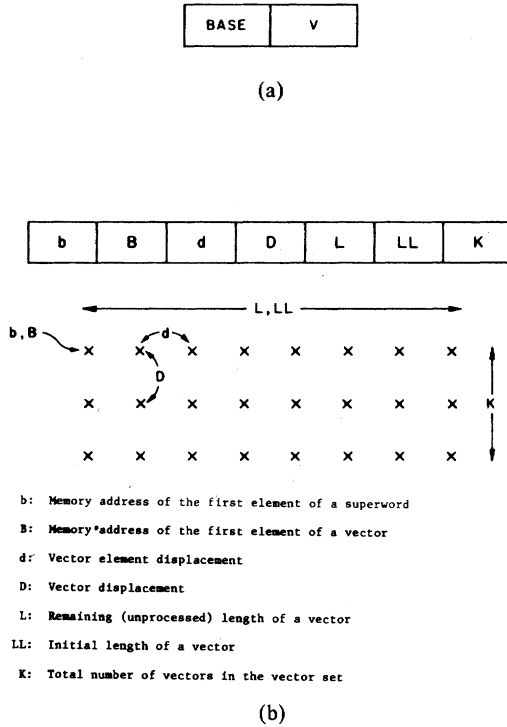


Fig. 3. (a) Array descriptor. (b) Vector set descriptor (VSD).

and alignment. This is greatly facilitated by special-purpose indexing hardware.

The purpose of the indexing hardware is to generate alignment tags and memory addresses for vector access. Consider first the input alignment network. To access a superword, processor p must generate an input alignment tag IAT, which specifies the memory module number of the p th element of the superword, i.e., $\mu(p)$. At the same time, the address of the p th element $\alpha(p)$ is generated in memory $\mu(p)$. Notice that each processor could generate the required address using (5), and then route this address to the proper memory through the output alignment network. However, by using (7) we avoid the extra routing operation.

The output alignment network works similarly. Memory $\mu(p)$ is to receive the p th element of a superword, and thus generates an output alignment tag OAT whose value is computed from (6) above. Each memory also computes the required address $\alpha(\mu)$ for storing the output.

The alignment, indexing, and memory systems are also responsible for a number of other functions as follows:

- 1) processor to processor routing, without going to memory,
- 2) handling of nonlinear P -vectors through the use of tags and addresses computed by the processor, or stored in special ROM memories or main memory,
- 3) handling bit vectors used for sparse vector operations and conditional vector expressions,
- 4) padding short superwords and with null elements,
- 5) providing for broadcast of scalar data from the control unit or main memory.

We will discuss these functions in a later section. For now we will restrict our attention to accessing linear P -vectors.

A. Linear P -Vector Access

Let us assume for the moment that we are interested in access to a single superword with initial base address B and with order d . If the superword is to be *fetched* from the memory, then for each memory μ , we must generate an address [see (4)–(7)]

$$\alpha(\mu) = \{B + p(\mu) \cdot d\} / P \quad (8)$$

where

$$p(\mu) = \{(\mu - B)d' \bmod M\} \quad (9)$$

and for each processor p , we must generate an IAT

$$\mu(p) = (B + d * p) \bmod M. \quad (10)$$

However, if the superword is to be *stored* in the memory, then for each memory μ we must generate an address given by (8) and (9) and for each memory μ we must also generate an OAT

$$p(\mu) = [(\mu - B)d'] \bmod M. \quad (11)$$

Thus, M – addresses and P – IAT's or M – OAT's are required to access a superword. In the next section we will show how the generation of these values can be simplified.

1) *Recursive Generation Technique*: Consider (10). Substituting $(p + k)$ and $(p - k)$ for p , we get

$$\begin{aligned} \mu(p \pm k) &= [B + d * (p \pm k)] \bmod M \\ &= [\mu(p \pm k \mp 1) \pm d] \bmod M. \end{aligned} \quad (12)$$

Equation (12) implies that $\mu(p + k)$ can be generated from $\mu(p)$ with modulo M addition/subtraction operations instead of a multiply followed by a modulo M addition. Extending the notion, from any $\mu(p)$ all tags can be generated recursively with appropriate modulo M additions or subtractions. In practice, primary $\mu(p)$ for several values of p are generated using (10), and secondary $\mu(p)$ for the remaining values of p are generated using (12). The number of primary $\mu(p)$ versus the number of secondary $\mu(p)$ calculated can be determined by a simple hardware versus time tradeoff.

The same technique can be applied to generate output alignment tags and memory addresses. The equation for the OAT's is

$$p(\mu \pm k) = [p(\mu \pm k \mp 1) \pm d'] \bmod M. \quad (13)$$

For memory addresses, the equation is

$$\alpha(\mu \pm k) = (B + \{[p(\mu \pm k \mp 1) \pm d'] \bmod M\}d) / P. \quad (14)$$

2) *BSP Implementation*: For the BSP, $P = 16$ and $M = 17$. The base address B is a 23-bit value. Element displacement d is a 23-bit signed quantity. For timing and hardware considerations, 4 initial memory addresses, 4 IAT's, and 4 OAT's are generated by using multiplications and modulo and normal additions. Other addresses and tags are generated by using binary adders. To use the binary adders, the equations de-

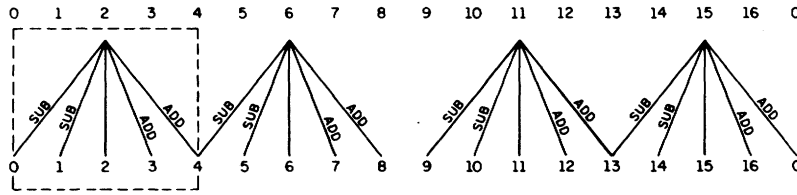


Fig. 4. Primary and secondary address generation.

scribed in the previous section were further simplified as follows. Let $\delta = d \bmod M$, and notice that $\mu(p) < M$. For IAT's, we get

$$\mu(p + k) = \mu(p) + k\delta - cM \quad (15)$$

where c is a correction factor such that $cM \leq \mu(p) + k\delta < (c + 1)M$. For example, assume that $M = 17$. We might generate primary $\mu(p)$ for $p = 1, 4, 7, 10, 13, 16$ from (10). Secondary $\mu(p)$ for the remaining values of p would be generated as follows from (15):

$$\mu(p + 1) = \mu(p) + \delta \text{ corrected by } -17 \text{ if } \mu(p) + \delta \geq 17$$

$$\mu(p - 1) = \mu(p) - \delta \text{ corrected by } +17 \text{ if } \mu(p) - \delta < 0$$

Equations for OAT's are the same as above except δ is replaced by d' . For memory address generation, the equations are as follows. Let

$$A(\mu) = B + p(\mu) \cdot d \text{ so that } \alpha(\mu) = A(\mu)/P.$$

Then

$$\begin{aligned} A(\mu + k) &= B + d \cdot p(\mu + k) \\ &= B + d([d'(\mu + k - B)] \bmod M) \\ &= B + d([p(\mu) + kd'] \bmod M) \\ &= A(\mu) + kdd' - dcM \end{aligned}$$

where

$$cM \leq p(\mu) + kd' < (c + 1)M. \quad (16)$$

Address generation in the BSP is performed as follows. Primary $A(\mu)$ are generated for $\mu = 2, 6, 11$, and 15 as shown in Fig. 4. Then secondary values are generated for $k = \pm 1, \pm 2$. Notice that $A(\mu)$ for $\mu = 4$ and 13 are each generated twice. This redundancy is used to check the hardware integrity by comparing duplicated values. (In addition, modulo 3 checks are performed on all additions to further verify hardware integrity.)

A primary $A(\mu)$ generator is shown in Fig. 5. $B \bmod M$ and d' are each 5-bit quantities (since $M = 17$) and are supplied by the Central Index Unit (to be discussed in the next section). The quantity $d'(\mu - B) \bmod M$ is supplied by a 1024×5 bit ROM. (The ROM contents differ for each primary μ .)

A secondary address generator for $A(\mu + 2)$ is shown in Fig. 6. Notice that in (16) a test is required to determine the quantity added to (or subtracted from) $A(\mu)$. This test depends on the quantity $p(\mu) + 2d' = d'(\mu - B) \bmod M + 2d'$ [see (16)]. $d'(\mu - B) \bmod M$ (available from the primary address generator) and d' are each 5-bit quantities and are used as address inputs to a 1024×2 ROM. The output of the ROM determines the test result and is used to multiply the necessary

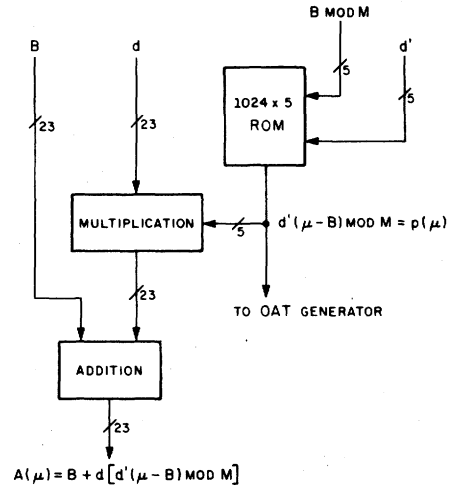


Fig. 5. A primary address generator.

additive factor for the final adder. The other secondary address generators for $A(\mu + 1)$, $A(\mu - 1)$, and $A(\mu - 2)$ are similar to the one shown in Fig. 6. However, the $A(\mu \pm 1)$ generators only need 2-way multiplexers and a one-bit wide decision ROM. (Through further simplification, these decision ROM's can be reduced to 512 words, so that the total decision ROM for four secondary generators is just 6×512 bits.) The primary and its four associated secondary address generators are all grouped together physically.

Generation of IAT's and OAT's is essentially the same or simpler than address generation. Only the values and number of bits change. One group of hardware, described above, generates the addresses, and a similar group of hardware generates both the IAT's and OAT's. Both groups of hardware form part of the Central Index Unit that will be described next.

3) *The Central Index Unit:* One of the components of the control unit is the Central Index Unit (CIU). The purpose of the CIU is to: 1) perform automatic indexing of multiple superwords of up to two-dimensional arrays, 2) generate input and output alignment tags, and 3) generate 4 initial memory addresses and indexing constants. The CIU can be divided into 4 major sections: 1) Descriptor Store Unit, 2) Descriptor Processing Unit, 3) IAT and OAT generators, and 4) Memory Address and Indexing Constant Generators. Fig. 7 shows the organization of the above sections. The IAT, OAT, and address generators were described in the previous section.

The descriptor store unit scores up to 16 vector set descriptors (VSD). A simplified descriptor's contents are shown in Fig. 3(b).

A superword access requires an Indexing Event in the CIU.

	b	B	d	D	L	LL	K
VSD-X	x	x	100	1	100	100	50
VSD-Y	y + 602	y + 602	300	604	100	100	50
VSD-Z	z + 202	z + 202	20001	402	100	100	50

(a)

	b	B	d	D	L	LL	K
VSD-X	x + 1600	x	100	1	84	100	50
VSD-Y	y + 5402	y + 602	300	604	84	100	50
VSD-Z	z + 320218	z + 202	20001	402	84	100	50

(b)

	b	B	d	D	L	LL	K
VSD-X	x + 1	x + 1	100	1	100	100	49
VSD-Y	y + 1206	y + 1206	300	604	100	100	49
VSD-Z	z + 604	z + 604	200	402	100	100	49

(c)

Fig. 8. (a) Initial VSD's. (b) VSD's after first superword. (c) VSD's after seventh superword.

to zero. The CIU performs the broadcast by making all tags equal to the memory module from which the scalar is to be broadcast.

2) *Scalar Data from the AE's*: There are some cases where scalar data produced by the AE are required by the CU. This case is handled by forcing the output alignment network to route the required data item to the CU.

3) *Vector Element Conflict*: In the memory storage scheme, if $d \bmod M = 0$, all the elements of the linear vector lie in the same memory module. This is referred to as a vector element conflict condition. In this case the access to the memory has to be sequential. In the BSP System this condition is handled by forcing superword size equal to 1. Thus, the BSP System automatically adapts to this case without any software or other interruption.

4) *Inner and Outer Loop Optimization*: Consider the following Fortran program segment.

```
DO 10 I = E, F, G
DO 10 J = P, Q, R
10  A(I,J) = B(I,J) + C(I,J)
```

This program can be performed in a single BSP vector operation using Vector Set Descriptors having $K = \left\lceil \frac{F-E}{G} \right\rceil + 1$ parallel vectors, each having a length equal to $L = \left\lceil \frac{Q-P}{R} \right\rceil +$

1. Clearly, the last superword length in each vector is equal to the remainder $L/16$. The remaining AE's will be idle. This will be repeated K times in the above case.

If the remainder $K/16$ is greater than the remainder $L/16$, it is faster to execute the above program segment with inner and outer loops interchanged. This is exactly what happens in the BSP to maximize performance. Hardware detection of the fastest loop order is made from the parameters L and K of a VSD. Of course, not all loops can be interchanged, and a software check is made to allow the above optimization.

C. Removal of "Holes" in the Memory System

The holes in the memory, generally, represent waste of the storage space. For the 17 memory BSP System, 1/17 of the total memory is not used, which was considered negligible. However, for other system configurations, this could be a significant amount. This section presents a technique to eliminate the holes.

Referring to (5) and (7), note that removal of holes means a divide by M function is required to determine the physical address for each memory module. Artzy *et al.* [1] presented an efficient method to perform divide by constant divisors. However, the method can be applied only if the remainder is zero.

Referring to (7), note that $A(\mu) \bmod M = \mu$. Thus, if we subtract μ from $\{d[(\mu - B)d' \bmod M] + B\}$, a zero remainder is guaranteed. Also, this subtraction does not change the quotient. Thus, the algorithm to efficiently divide by M is a subtraction by μ followed by a multiplication by a predetermined number, followed by taking the 2's complement of the result.

In addition to this, the simplification schemes discussed in earlier sections can be applied, thus making the "hole-less" memory system more efficient.

IV. CONCLUSION

In this paper we have shown one design for a conflict-free array access memory. This design is based on the use of a prime number of memories. Crucial to this design is the simplification of the indexing equations which allow most of the mod M operations and much of the other index calculations to be done with ROM's and other simple hardware. These simplifications were discussed in Section III, along with a brief discussion of some of the necessary indexing hardware. Further details can be found in [12].

The design of this memory system fits nicely in the context of the Burroughs Scientific Processor [19], [9]. The vector machine instructions on this computer can encompass two levels of loop nesting, and the indexing hardware carries out the necessary addressing and alignment calculations automatically, once the initial vector set descriptors have been set up. One of the major problems with large vector computers has been that indexing overhead and memory access conflicts have a significant effect on overall vector performance. By using the prime memory system and indexing hardware described in this paper, the BSP is able to execute vector instructions efficiently.

The performance of this SIMD machine depends to a large extent on its ability to access arrays in a deterministic, conflict-free way. Future supercomputers will undoubtedly tend more toward MIMD operation. In MIMD machines we must depend more on the probabilistic behavior of the memory system. That is, we must be concerned less with deterministic array access and more on the average time required to deliver data. Other work [4], [15], [6], [5] has shown that memory and communication schemes can be designed for use in MIMD

environments. Supercomputers will have to depend on a combination of parallel, pipeline, and MIMD techniques, together with complex memory systems.

ACKNOWLEDGMENT

A number of people have contributed to this work. The overall design of BSP is due primarily to D. Kuck and R. Stokes. Special thanks go to R. Cantarella, who made many significant contributions to the BSP design and especially to the alignment, indexing, and memory system. O. Gupta and R. Gupta also made contributions.

REFERENCES

- [1] E. Artzy, J. A. Hinds, and H. J. Saal, "A fast division technique for constant divisors," *Commun. Ass. Comput. Mach.*, vol. 19, pp. 98-101, Feb. 1976.
- [2] K. E. Batchner, "The multidimensional access memory in STARAN," *IEEE Trans. Comput.*, vol. C-26, pp. 174-177, Feb. 1977.
- [3] P. Budnik and D. J. Kuck, "The organization and use of parallel memories," *IEEE Trans. Comput.*, vol. C-20, pp. 1566-1569, Dec. 1971.
- [4] D. Chang, D. J. Kuck, and D. H. Lawrie, "On the effective bandwidth of parallel memories," *IEEE Trans. Comput.*, vol. C-26, pp. 480-490, May 1977.
- [5] P.-Y. Chen, D. H. Lawrie, D. A. Padua, and P.-C. Yew, "Interconnection networks using shuffles," *IEEE Computer*, vol. 14, pp. 55-64, Dec. 1981.
- [6] D. M. Dias and J. R. Jump, "Packet switching interconnection networks for modular systems," *IEEE Computer*, vol. 14, pp. 43-53, Dec. 1981.
- [7] T.-Y. Feng, "Data manipulation functions in parallel processors and their implementations," *IEEE Trans. Comput.*, vol. C-23, pp. 309-318, Mar. 1974.
- [8] D. D. Gajski and L. R. Rubinfield, "Design of arithmetic elements for Burroughs Scientific Processor," in *Proc. 4th Symp. Comput. Arithmetic*, 1978, pp. 245-256; also in *Proc. 1978 LASL Workshop Vector and Parallel Processors*, 1978.
- [9] D. J. Kuck and R. Stokes, "The Burroughs Scientific Processor (BSP)," *IEEE Trans. Comput.*, vol. C-31, this issue, pp. 363-376.
- [10] T. Lang, "Interconnections between processors and memory modules using the shuffle-exchange network," *IEEE Trans. Comput.*, vol. C-25, pp. 496-503, May 1976.
- [11] T. Lang and H. S. Stone, "A shuffle exchange network with simplified control," *IEEE Trans. Comput.*, vol. C-25, pp. 55-65, Jan. 1976.
- [12] D. H. Lawrie and C. R. Vora, "Multidimensional parallel access computer memory system," U.S. Patent 4 051 551, Sept. 27, 1977.
- [13] D. H. Lawrie, "Access and alignment of data in an array processor," *IEEE Trans. Comput.*, vol. C-24, pp. 1145-1155, Dec. 1975.
- [14] S. E. Orcutt, "Implementation of permutation functions in ILLIAC IV-type computers," *IEEE Trans. Comput.*, vol. C-25, pp. 929-936, Sept. 1976.
- [15] J. H. Patel, "Performance of processor-memory interconnections for multiprocessors," *IEEE Trans. Comput.*, vol. C-30, pp. 771-780, Oct. 1981.

- [16] H. D. Shapiro, "Theoretical limitations on the use of parallel memories," Ph.D. dissertation, Dep. Comput. Sci., Univ. of Illinois, Urbana-Champaign, Rep. No. 75-776, Dec. 1975.
- [17] H. J. Siegel, "Controlling the active/inactive status of SIMD machine processors," in *Proc. 1977 Int. Conf. Parallel Processing*, Aug. 1977, p. 183.
- [18] —, "Interconnection networks for SIMD machines," *IEEE Computer*, vol. 12, pp. 57-65, June 1979.
- [19] R. A. Stokes, "Burroughs Scientific Processor," in *High Speed Computer and Algorithm Organization*. New York: Academic, 1977, pp. 85-89.
- [20] H. S. Stone, "Parallel processing with the perfect shuffle," *IEEE Trans. Comput.*, vol. C-20, pp. 153-161, Feb. 1971.
- [21] R. C. Swanson, "Interconnections for parallel memories to unscramble p -ordered vectors," *IEEE Trans. Comput.*, vol. C-23, pp. 1105-1115, Nov. 1974.



Duncan H. Lawrie (S'66-M'73-SM'81) is currently an Associate Professor of Computer Science at the University of Illinois at Urbana-Champaign. He has been active in the area of parallel computation systems since 1966, when he worked on the Illiac IV computer. Since then he has contributed to the design of several other machines, including most recently the Burroughs Scientific Processor. He specializes in research and consulting in the area of design, construction, and use of very large computing systems.

Dr. Lawrie has served as a referee for the IEEE TRANSACTIONS ON COMPUTERS as well as several national conferences. He was Program Co-Chairman for the Symposium of High-Speed Computer and Algorithm Organization, the 9th International Conference on Parallel Processing, a member of the Program Committee for the Workshop on Interconnection Networks, and Program Co-Chairman for the 1980 International Conference on Parallel Processing. He is a member of Tau Beta Pi, Eta Kappa Nu, Sigma Xi, the Association for Computing Machinery, and the American Association of University Professors.



Chandra R. Vora (S'76-M'78) received the B.S.E.E. degree from M.S. University of Baroda, Baroda, India, and the M.S. degree in electrical sciences from the State University of New York in 1973.

Currently, he is a Project Engineer in the Technical Products Division of Data General Corporation, Westboro, MA. Previously, he was with the Burroughs Corporation, where he was involved in the design and development of the Burroughs Scientific Processor. His interests include parallel distributed processing, multiprocessors systems, and microprocessors. He holds several U.S. patents in the computer system design field.